AGSTU	TEKNISK RAPPORT	AGSTU
Författare: Darwin Rengifo E-postadress :	Uppgift: TEIS ECS – Embedded Computer System Fil: darwin_rengifo_HWSW_task_7 Version : 1	

HW/SW PROTOTYP AV REAL-TIME KERNEL(RTK) SYSTEM

Darwin Rengifo

2020-07-18

Sammanfattning:
Denna rapport beskriver en förstudie och planering för Multi-tasking med användningen av Sierra Real-Time Kernel(RTK-accelerator), DE10-Lite kortet, brytare för att testas Multi-tasking jobb, tre lysdioderna från kortet som visar Multi-tasking aktivitet. Denna rapport innehåller aktivitets- och tidplan, projektkostnad, kravspecifikations, projektarkitektur och beskrivning av utvecklade komponenter.

Innehåll

1. INLEDNING 2
2. KRAVSPECIFIKATION FRÅN KUND 3
3. TASK OCH SCHEMALÄGGARE 4
3.1 Task 4
3.2 Schemaläggare 4
3.3 Hårvaran arkitektur för ingående komponenter 4
3.4 Systemet kan byggas större 4
3.5 Systemarkitektur 5
4. MJUKVARAN ARKITEKTUR 8
4.1 Minneskarta 8
4.2 Programarkitektur 8
4.3 Utveckling av systemet med Real-Time Kernel(Sierra-RTK) 9
5. PULSDIAGRAM ANALYS UNDER 5 SEKUNDER 10

1. INLEDNING

Denna rapport beskriver hur man bygger ett ”Multi-tasking System”. Konstruktionen ska vara uppdelad på fem tasking och en Sierra Ral-Time Kernel(RTK-accelerator) för att exekverar tasking.

2. KRAVSPECIFIKATION FRÅN KUND

Tabell 1. Kravspecifikation från kund

Krav	Beskrivning	Utfört Ja/nej
	Förstudie Krav	
1	Definiera begreppen task och schemaläggare. (eget kapitel) HW arkitektur för att minst uppfylla kraven, kan byggas större (eget kapitel)	Ja
	Funktionella krav	
2	Konstruera ett inbyggt system (HW och SW) som har tre oberoende task och ideal task. Koppla in tryckknapp och valfria lysdioder. Hårdvaran uppdateras vid behov. **Task_A**: (mellan prioritet) Ska blinka och skriva ut ett ”**A**” på terminalfönstret varje sekund. Om KEY[0] trycks in, ska den inte skriva ut på terminalfönstret. **Task_B:** (högsta prioritet): Ska skriva ett ”**B**” ut på terminalfönstret två gånger i sekunden. **Task_C:** (näst lägst prioritet) Ska skriva ut ett ”**C**” på terminalfönstret var annan sekund. **Ideal:** (lägst prioritet) Är bara en oändlig loop med en intern lång loop. Skriver ut en punkt. Se bilagan. En figur med programarkitektur, en kort beskrivning av arkitekturen och C koden i ett eget kapitel. Beskriv de Sierra-anrop som används	Ja
2B	Beskriv kort konfigureringen av RTK med nedanstående task (eget kapitel)	Ja
	Analys krav	
3	Rita ett pulsdiagram under 5 sekunder och beskriv kort vad som händer. Konstruktören får själv rita pulsdiagrammet. Lägg in diagrammet i ett eget kapitel med en kort beskrivning.	Ja
	Andra krav	
4	Krav på konstruktionskoden 1) Filhuvud i varje fil 2) Kommentera koden	Ja
	Leverans krav	
5	Leveransen ska ske till plattformen Itslearning. Leveransen ska vara en rapport med det som krävs från kunden. Namnet på filen ska vara ”fornamn_efternamn_HWSW_task_7”.	Ja

3. TASK OCH SCHEMALÄGGARE

3.1 Task

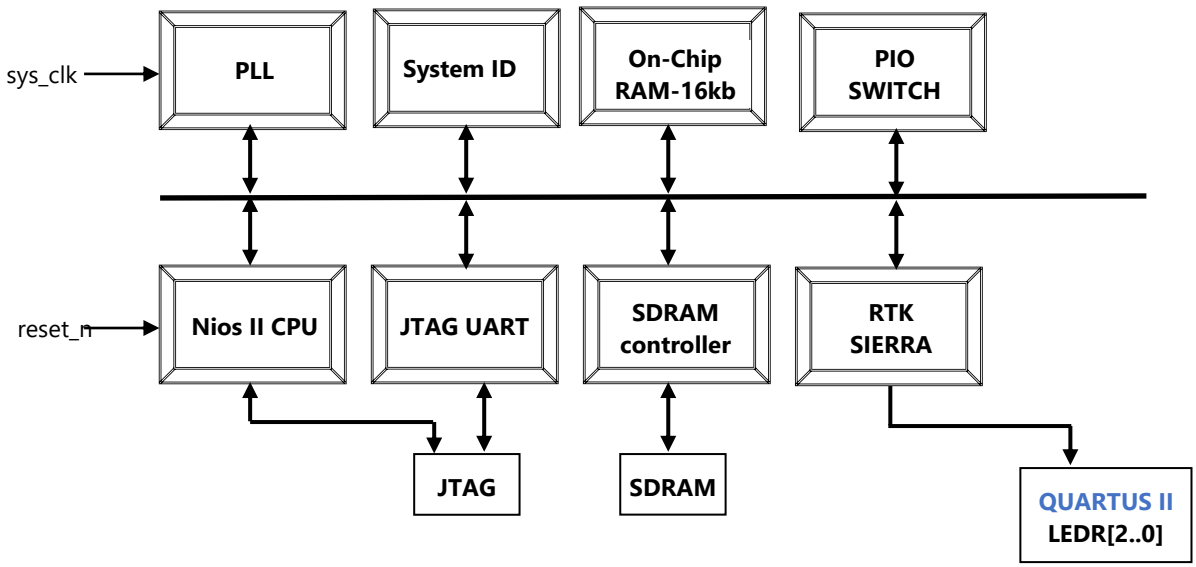
Dessa funktioner(tasking) anropas inte på någonstans i programmet, de skapas bara. Varje task exekverar kontinuerligt och oberoende sätt som kan tävlar med andra samtidiga task för processorns exekvering tid. Den Kernel är ansvarar för hanteringen av alla tasking.

3.2 Schemaläggare

Den schemaläggare bestämma vilken task det kommer att börjat att exekverar som baserar det på en mekanism av tilldelning prioriteten som kan specificerar på programmet.

3.3 Hårvaran arkitektur för ingående komponenter

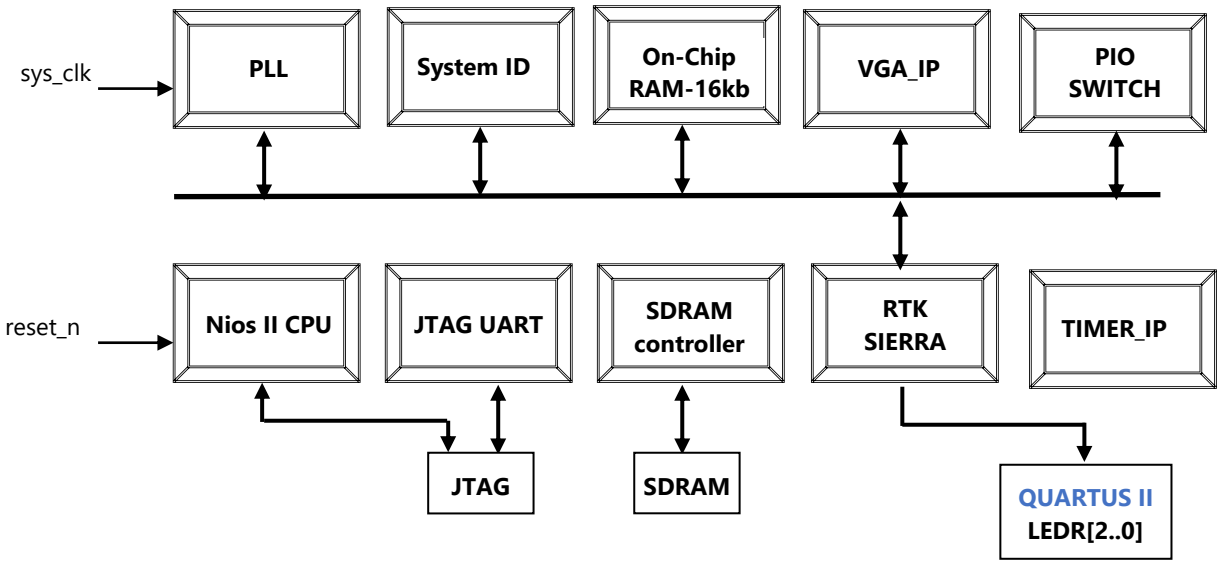
Systemarkitekturen som FPGA:n programmerats med Sierra Real-Time Kernel-RTK(accelerator) för att hanterar tasking på systemet. Quartus koppla in tre pinnar till RTK-sierra external pinnar för att identifierar exekvering task. Se Figur 1.



Figur 1. Systemarkitektur, blockschema för MULTI-TASKING

3.4 Systemet kan byggas större

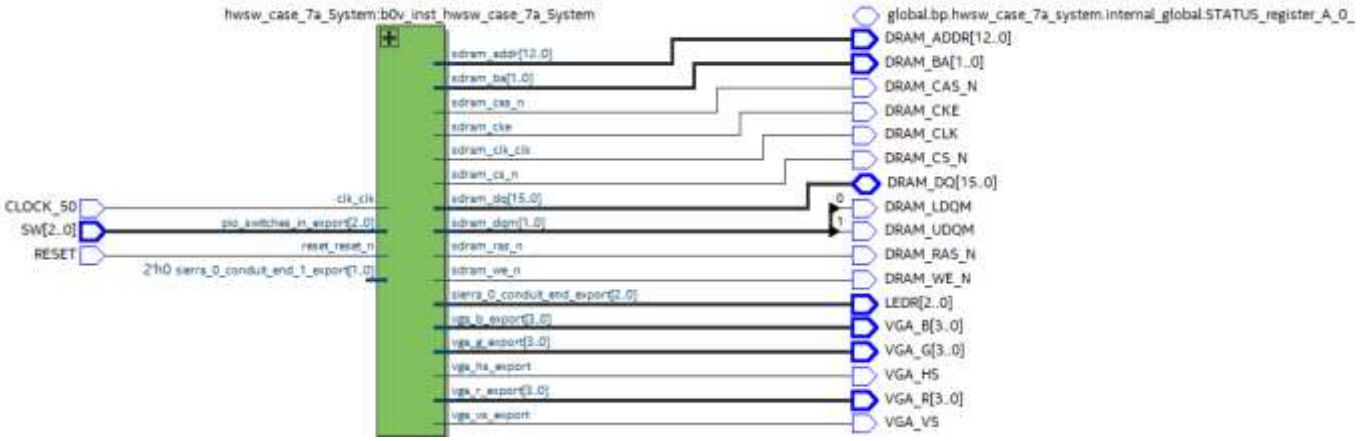
Systemarkitekturen som FPGA:n programmerats med Sierra Real-Time Kernel-RTK(accelerator) har två extra komponenten för att framtida konstruktionen, en VGA_IP för att visa resultaten på skärmen beroende på task prioritetet och en TIMER_IP som exekverar sig från hårdvaran för att kontrollerar förseningar tiden inom systemet. Se Figur 2.



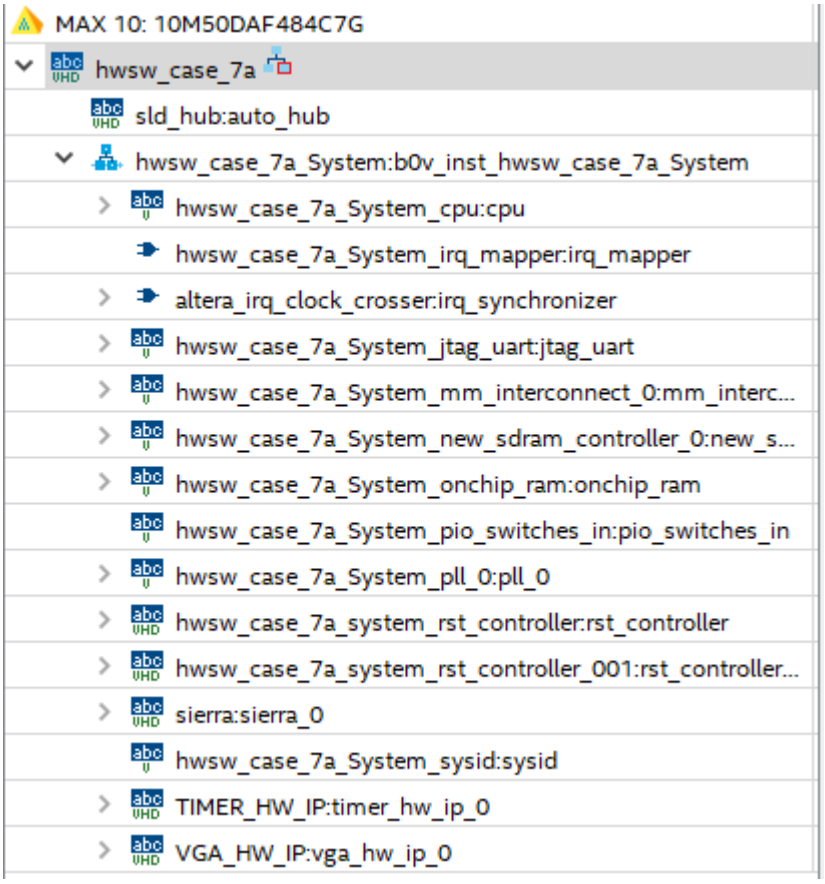
Figur 2. Systemarkitektur, blockschema för bygg större MULTI-TASKING

3.5 Systemarkitektur

TOPPNIVÅN arkitektur visas i Figur 3, i figur 4 visar komponenthierarki och i figur 5 visar QSYS interface.



Figur 3. Systemarkitekturen för för MULTI-TASKING SYSTEM



Figur 4. Komponenthierarki för för MULTI-TASKING SYSTEM



Figur 5. QSYS interface för MULTIPROCESSOR SYSTEM

4. MJUKVARAN ARKITEKTUR

4.1 Minneskarta

Memory-map från ”sumary.html” för konstruktionen, se figur 6.

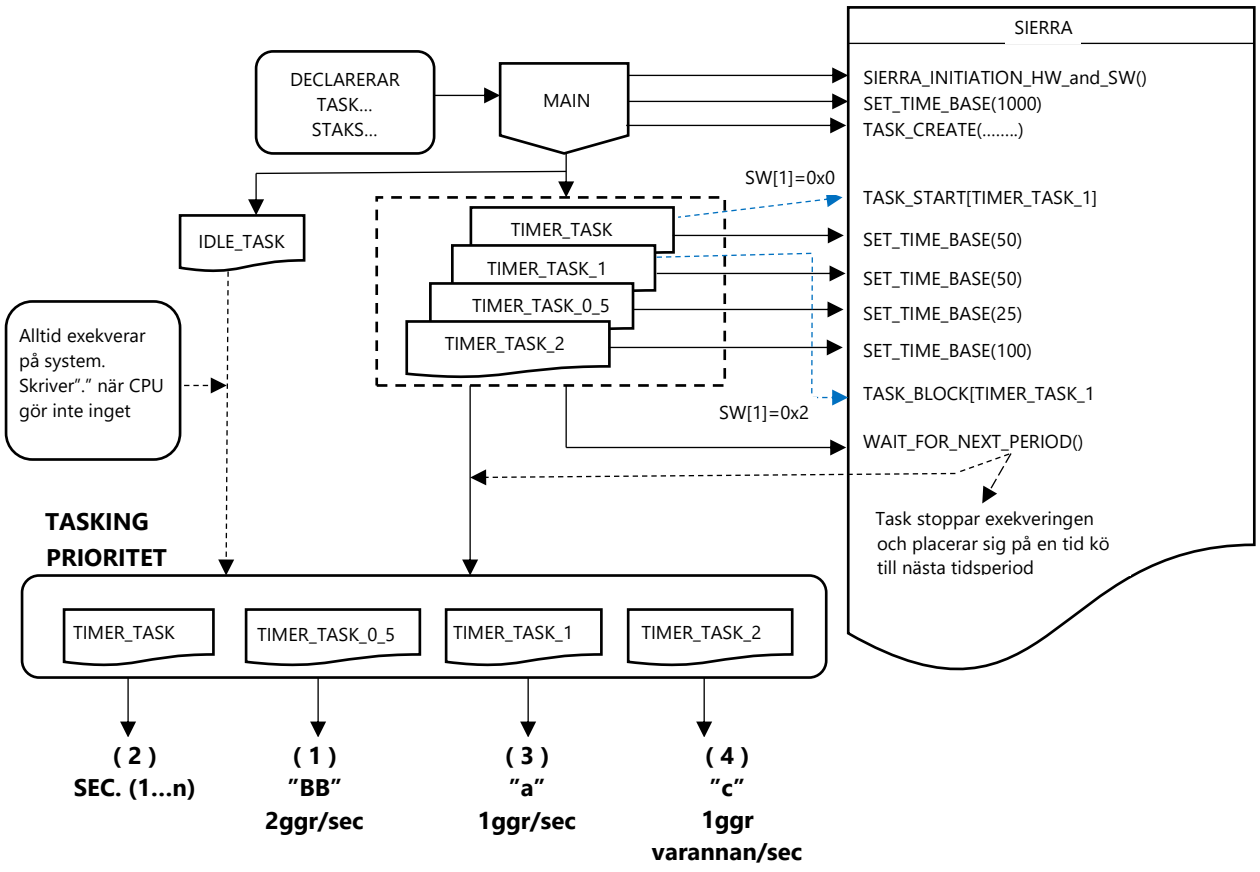
Nios II Memory Map

Slave Descriptor	Address Range	Size	Attributes
jtag_uart	0x08089438 - 0x0808943F	8	printable
sysid	0x08089430 - 0x08089437	8	
TIMER_HW_IP_0	0x08089420 - 0x0808942F	16	
pll_0	0x08089410 - 0x0808941F	16	
pio_switches_in	0x08089400 - 0x0808940F	16	
sierra_0	0x08089000 - 0x080893FF	1024	
onchip_ram	0x08084000 - 0x08087FFF	16384	memory
VGA_HW_IP_0	0x08000000 - 0x0807FFFF	524288	
new_sdram_controller_0	0x04000000 - 0x07FFFFFF	67108864	memory

Figur 6. Flödesdiagram MULTI-TASKING

4.2 Programarkitektur

I figur 7 visar C program arkitekturen med exekvering task.



Figur 7. C Arkitektur för MULTI-TASKING – SIERRA(RTK)

4.3 Utveckling av systemet med Real-Time Kernel(Sierra-RTK)

Till det här projektet det använder RTK-sierra accelerator son hanterar ”tasking” på CPU.

RTK-Sierra har ett set_timebase() för att ställer in tidsbasen interna av den klockan för Sierra. Med Sierra task_create() har skapat ”tasking” för det projekt med parameter som prioritet, ”blocked eller ready” tillstånd för att kan hanterar vilken task ska exekverar först och med vilken prioritet.

Inuti varje task som har skapat har det definierat sig en tidsperiod t.ex init_period_time(25) som Initiera tidsperioden för anropar ”tasking”. Det har definierat sig också en wait_for_next_period() för att för varje anropat till denna funktion tasken stoppar exekveringen och placerar sig på en tid kö. När tidsperioden för en task har gått, real-time kernel det kommer att starta om denna task.

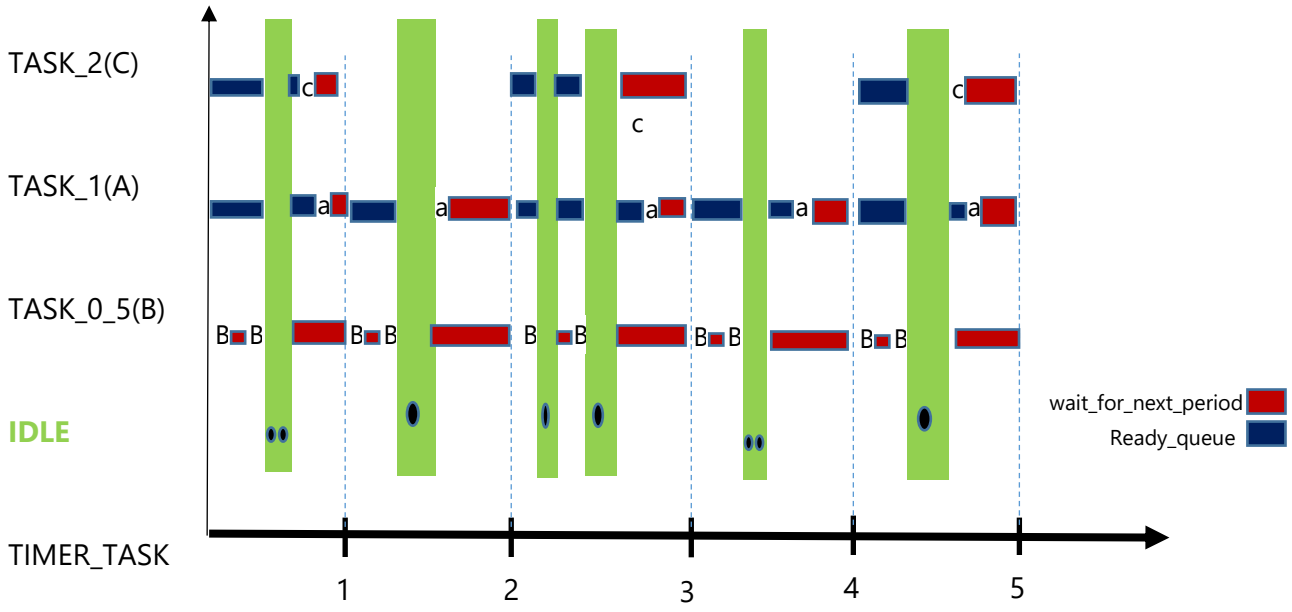
Därtill har det definierat sig en linje av information med "if (test.periodic_start_integer & 0x1)" för att verifierar om nuvarande ”task” hinner inte exekverar nästa period om detta händer, det kommer att visa en meddelande som kommer att indikerar att period är slut för att exekverar tasken men försätta med nästa task i alla fall.

5. PULSDIAGRAM ANALYS UNDER 5 SEKUNDER

I FIGUR 9 visar det exekveringen under 5 sekunder och I FIGUR 10 visar en pulsdiagram och ett kort beskrivning som visar vad som händer med ”tasking” under 5 sekunder.

```
 Sierra HW version = -1809841405
 Sierra SW driver version = 100315
Timer Task 2 init    => c
Timer Task 1 init    => a
Timer Task init
Timer Task 0,5 init  => BB
.B..a
Sec: 1 BB..ca
Sec: 2 BB.a
Sec: 3 B.B.ca
Sec: 4 BB..a
Sec: 5 BB.ca
```

Figur 8. Exekveringen av MULTI-TASKING – SIERRA(RTK)



Figur 9. Pulsdiagram av MULTI-TASKING – SIERRA(RTK)

Pulsdiagram visar exekvering task under 5 sekunder:

Sekund 1:

IDLE: Det exekverar alltid och skriver en poäng när CPU gör inget och väntar att exekverar en task.

Task_0_5: Väntar halv sekund(röda linje) och placerar sig på en tids kö(blå linje) efter slutar tiden skriver ut ”B” på konsolen och gör samma sak för att exekverar nästa tidsperiod. Den tasken kommer det att exekverar alltid förs eftersom har högsta prioritet.

Task_1: Väntar en halv sekund(röda linje) och placerar sig på en tids kö(blå linje) efter slutar tiden skriver ut ”A” på konsolen om inte finns andra task med hösta prioritet och gör samma sak för att exekverar nästa tidsperiod. Den tasken kommer det att exekverar alltid som andra eftersom har det mellan prioritet.

Task_2: Väntar två halv sekund(röda linje) och placerar sig på en tids kö(blå linje) efter slutar tiden skriver ut ”C” på konsolen om inte finns andra task med hösta prioritet och gör samma sak för att exekverar nästa tidsperiod. Den tasken kommer det att exekverar alltid som andra eftersom har det näst lägst prioritet.

För att slutar man kan säga att den IP Sierra Real-Time Kernel(accelerator) på CPU´n har jobbet av hanterar alla tasking beroende på prioriteten och tidsperiod som anvisar sig på varje task. Ett intressant del är att när task inte hinner på tiden för att exekverar nästa tidsperiod den RTK återfår task exekvering under följande tidsperioder.